AI-Based Fire and Smoke Detection System Using Deep Learning for Real-Time Emergency Monitoring

M R Varun
Department of Computer Science and Engineering
Vidyavardhaka College of Engineering
Mysuru, India
vvce23cse0254@vvce.ac.in

M H Mayur Gowda
Department of Computer Science and Engineering
Vidyavardhaka College of Engineering
Mysuru, India
vvce23cse0077@vvce.ac.in

Kedar Lokesh Vandith
Department of Computer Science and Engineering
Vidyavardhaka College of Engineering
Mysuru, India
vvce23cse0169@vvce.ac.in

Dr. Janhavi V
Associate Professor
Department of Computer Science and Engineering
Vidyavardhaka College of Engineering
Mysuru, India
janhavi.v@vvce.ac.in

Dr. Ayesha Taranum
Associate Professor & Deputy HOD
Department of Computer Science and Engineering
Vidyavardhaka College of Engineering
Mysuru, India
ayesha.cs@vvce.ac.in

***Abstract*—Fire outbreaks in residential, industrial and public infrastructure remain a leading cause of property loss and casualties, and the delay between ignition and human-observed detection is often the largest contributor to escalation. Point-type smoke and heat sensors are inexpensive but slow in open or high-ceiling spaces and cannot localise an event, while manual monitoring of CCTV feeds is unreliable over long shifts. This paper presents a vision-first fire and smoke detection system that operates on existing camera infrastructure and converts detections into verified alerts within seconds. The pipeline combines a stratified two-source fire and smoke corpus (indoor and outdoor, 4,200 annotated images), a YOLOv11n single-stage detector fine-tuned from COCO weights, a temporal decision engine with separate warning, confirmation and clearing thresholds, optional sensor fusion with a low-cost IoT gas and temperature node, evidence capture, multi-channel notification, and a web dashboard backed by a REST API and an event database. On a held-out test split of 600 images the detector reaches mAP@0.5 of 0.484 (fire 0.461, smoke 0.508) with precision 0.519 and recall 0.490, and runs at 31.4 frames per second on a laptop CPU without a GPU. The temporal engine suppresses single-frame false alarms and is verified by unit tests and live fail-safe scenarios (camera loss, sensor disconnect, network loss, emergency stop). Limitations under dense smoke, steam, fog and low light are discussed together with a roadmap toward edge deployment and multi-camera operation.**

***Index Terms*—Fire detection, smoke detection, deep learning, YOLOv11, real-time monitoring, computer vision, emergency response, IoT, CCTV surveillance**

I. Introduction

Fire-safety engineering has historically relied on point-type detectors—ionisation or photoelectric smoke alarms and fixed-temperature heat sensors—that respond to the physical by-products of combustion reaching the sensor itself. These devices are dependable in enclosed rooms but degrade sharply in large, open or high-ceiling environments such as warehouses, atriums, parking structures and outdoor perimeters, where smoke disperses before reaching a sensor and where the sensor gives no indication of the fire's location, size or growth rate. As buildings are increasingly instrumented with CCTV networks for security, a natural complementary strategy has emerged: use the cameras that are already installed to recognise fire and smoke visually, rather than waiting for particulate matter to travel to a fixed point.

Vision-based fire recognition is not new, but it has undergone a qualitative shift over the last five years. Early approaches relied on hand-crafted colour and motion cues—flame-coloured pixel clustering, flicker analysis and optical-flow smoke growth—which were cheap but fragile, producing false positives for orange lighting, sunsets and headlights and false negatives for translucent or night-time smoke. Deep convolutional networks and, more recently, single-stage detectors of the YOLO family [1], [10], [11], Vision Transformers [8] and hybrid CNN–Transformer models have substantially closed this gap by learning discriminative representations directly from data.

Detection alone, however, is not a safety system. A raw bounding box cannot be operationalised by an emergency workflow without confidence filtering, temporal confirmation, alert generation, evidence capture and a monitoring interface, and most published detectors stop at the per-frame metric. This paper targets real-time, CCTV-native fire and smoke detection for emergency monitoring and makes the follow-

ing contributions: (i) a domain-specific YOLOv11n detector trained on a stratified indoor–outdoor corpus with explicit negative images, documented with a dataset register and reproducible configuration; (ii) a temporal decision engine with hysteresis and optional sensor fusion that converts per-frame detections into NORMAL, WARNING, CONFIRMED and RECOVERY states and never triggers on a single frame; (iii) a complete operational pipeline—evidence snapshots, event database, REST API, notifications and live dashboard—with fail-safe behaviour under camera, sensor and network loss; and (iv) a measured evaluation on held-out data including per-class metrics, false-alarm analysis and CPU latency. The system is an academic prototype and is not presented as certified life-safety equipment.

The remainder of the paper is organised as follows. Section II surveys recent work. Section III states the problem and objectives. Section IV describes the dataset, detector and decision engine. Section V presents the system architecture, workflow and algorithm. Section VI reports results. Section VII discusses limitations and future work, and Section VIII concludes.

II. Literature Review

Recent research can be grouped into four strands, summarised in Table I.

TABLE I
Representative recent work on deep-learning fire and smoke detection.

Method	Core technique	Year	Focus
YOLOv8/v7 fine-tuned [12]	single-stage detector	2024	wildfire smoke, flame
Improved YOLOv8s [13]	light backbone + attention	2024	coal-mine fire
Improved YOLOv5s [14]	feature-fusion refinement	2024	general fire
FS-YOLO [15]	joint fire–smoke head	2024	fire and smoke
YOLOv11 [11]	C3k2, SPPF, C2PSA	2024	general detection
Shallow guide deep [17]	lightweight CNN	2024	early-stage fire
CNN+Transformer [18]	hybrid	2024	multiscale fire
Fire-ViT [19]	Vision Transformer	2025	tunnel fire
CN2VF-Net [20]	hybrid CNN–ViT	2025	complex scenes
FireNet-KD [21]	Swin + distillation	2025	wildfire
SmokeSeger [24]	Transformer–CNN seg.	2023	urban smoke
CAS-ViT [25]	additive attention ViT	2024	mobile vision
This work	YOLOv11n + temporal engine + IoT fusion	2026	CCTV emergency monitoring

A. YOLO-Family Detectors

YOLOv8 has become a common baseline because of its anchor-free head and efficient backbone [10]. Saleh *et al.* fine-tuned YOLOv8 and YOLOv7 for wildfire smoke and flame, reporting that adaptation of the neck and head improved recall on thin early smoke [12]. Kong *et al.* proposed an improved YOLOv8s for coal-mine monitoring that cut parameters and FLOPs by roughly a quarter while adding attention to recover accuracy, a pattern directly relevant to edge deployment [13]. Dou *et al.* refined YOLOv5s feature fusion [14], Wang *et al.* proposed FS-YOLO for joint fire–smoke detection [15], and Khanam and Hussain documented the YOLOv11 changes—the C3k2 block, updated SPPF and the C2PSA attention block—which improve small-object recall and speed over YOLOv8 [11].

B. CNN and Lightweight Backbones

Before single-stage detectors dominated, CNN classifiers were used for frame-level fire/no-fire decisions. Li *et al.* proposed a shallow-guide-deep network prioritising low latency for early-stage fire [17]; Wu *et al.* combined CNN features with a Transformer encoder for multiscale flames [18]. Depthwise-separable designs such as Xception [9] and MobileNet [6], and compound scaling in EfficientNet [7], remain references for resource-constrained cameras.

C. Vision Transformers and Hybrids

ViT-based models [8] capture the diffuse, non-rigid boundaries of smoke that a purely convolutional receptive field may fragment. Liu and Mariano's Fire-ViT reports strong accuracy in uniformly lit tunnels but reduced robustness outside that domain [19]. Ahmad *et al.* proposed CN2VF-Net and FireNet-KD, hybrid and Swin-based frameworks using knowledge distillation to keep inference tractable [20], [21]; Jing *et al.*'s SmokeSeger addresses urban smoke segmentation [24]; CAS-ViT makes additive self-attention mobile-friendly [25].

D. System-Level Work and the Gap

A growing number of works report FPS alongside accuracy, reflecting a shift toward evaluating fire detectors as components of a monitoring pipeline [22], [24]. Yet the reviewed literature rarely specifies how a detection becomes a verified alert, how false alarms from steam, fog and lighting are suppressed over time, or how the system behaves when a camera or sensor fails. These operational questions are the focus of this work.

III. Problem Statement and Objectives

Three practical gaps limit the deployment of fire recognition as a dependable real-time layer over existing CCTV. First, many detectors are evaluated on curated benchmarks rather than on noisy, variably lit footage from installed cameras, so reported accuracy does not always transfer. Second, models that gain accuracy by adding depth or Transformer blocks often pay in latency, which is unacceptable when seconds matter. Third, detection alone is not a safety system: without confirmation, alerting and logging, a bounding box cannot be acted upon. The objectives of this work are therefore to:

1) design and implement a real-time detection pipeline built on a lightweight YOLOv11 backbone that runs on commodity hardware;

TABLE II
DATASET DETAILS (STRATIFIED SUBSET, SEED 42).

Split	Images	Fire	Smoke	Both	Neg.	Boxes F / S
Train	3,000	988	988	723	301	2,873 / 2,256
Validation	600	189	187	164	60	551 / 487
Test	600	203	202	135	60	582 / 445
Total	4,200	1,380	1,377	1,022	421	4,006 / 3,188

2) construct a stratified dataset and augmentation strategy that improves robustness to scale, illumination and scene variation, with explicit negative images;
3) develop a temporal confirmation and alert module that converts raw detections into de-duplicated, evidence-backed emergency notifications;
4) provide a backend API and web dashboard for live monitoring, acknowledgement, history and device health;
5) evaluate with precision, recall, F1, mAP@0.5, mAP@0.5:0.95, false-alarm rate on negatives and CPU latency; and
6) identify operational limitations and a roadmap toward edge and multi-camera deployment.

IV. PROPOSED METHODOLOGY

A. Dataset

Two public YOLO-format corpora were combined so that both outdoor CCTV scenes and indoor flames are represented: a 40,900-image wildfire and CCTV smoke collection (CC-BY-4.0) [28] and the 7,370-image indoor ISFire-4 set (MIT) [29], whose four fine classes (black smoke, blue fire, white–grey smoke, yellow–orange fire) were mapped to the two target classes *fire* and *smoke*. Because training was performed on a CPU, a stratified subset was drawn from the upstream train/validation/test partitions so that frames from one source video never cross splits. Within each split, images were sampled round-robin from the fire-only, smoke-only and both-present groups, with up to 15 % negative images (scenes without fire or smoke) retained as hard negatives. Table II gives the resulting distribution; a dataset register records source, licence, class mapping and sampling seed.

B. Preprocessing and Augmentation

Frames are letterboxed to a square input of 416×416 pixels (preserving the aspect ratio of elongated flames and plumes) and scaled to $[0, 1]$. Training augmentation follows the Ultralytics pipeline: HSV perturbation ($h = 0.015$, $s = 0.6$, $v = 0.5$) to simulate white-balance and lighting change, horizontal flip ($p = 0.5$), scale jitter ($\pm 50\,\%$), translation ($\pm 10\,\%$), rotation ($\pm 5^\circ$) and mosaic composition, which forces the network to learn flame and smoke at several scales within one image. Mosaic is disabled for the final epochs so that the detector converges on natural image statistics.

C. Detector

The recognition engine is YOLOv11n [11], the smallest variant of the family (2.6 M parameters), chosen because the requirement is live inference on a laptop CPU. Its anchor-free head and C2PSA attention block help with diffuse, low-contrast smoke. The backbone is initialised from COCO-pretrained weights and fine-tuned end-to-end on the corpus of Table II. Each prediction is a tuple (class, confidence, box). Detection quality is measured with the intersection over union between a predicted box B_p and a ground-truth box B_g,

$$\mathrm{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}, \tag{1}$$

with a prediction counted as correct when $\mathrm{IoU} \geq 0.5$ and the class matches. Precision, recall and F1 follow from the resulting true positives (TP), false positives (FP) and false negatives (FN):

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}, \tag{2}$$

and mAP@0.5 is the mean over classes of the area under the precision–recall curve at the 0.5 IoU threshold.

D. Temporal Decision Engine

Raw detections are noisy: a reflection or a torch may produce one high-confidence box, and a genuine flame may drop below threshold for a few frames. The decision engine therefore reasons over a sliding time window W (3 s) of per-frame best detections (t_i, c_i). A frame is a *possible* hit when $c_i \geq \tau_w$ (0.35) and a *strong* hit when $c_i \geq \tau_c$ (0.55). The system enters WARNING when at least $N_w = 3$ possible hits fall inside W, and CONFIRMED when

$$\left|\{i : t_i \in W,\ c_i \geq \tau_c\}\right| \ \geq\ N_c \quad \text{and} \quad t_{\text{last}} - t_{\text{first}} \ \geq\ T_{\min}, \tag{3}$$

with $N_c = 10$ and $T_{\min} = 2\,\text{s}$, so that a single unstable frame can never trigger a response. Clearing uses a lower threshold $\tau_{\text{clear}} = 0.25$ and a hold time of 4 s (hysteresis), after which CONFIRMED moves to RECOVERY and returns to NORMAL after a 10 s stable period; a re-appearance during RECOVERY re-confirms immediately. When a sensor node reports elevated gas or temperature the required strong-hit count is lowered to $N_c' = 6$ (fusion), and persistent sensor evidence without any visual target raises a separate SENSOR_WARNING. Readings older than 15 s are discarded, so a disconnected sensor cannot influence the decision. All thresholds are stored in a configuration file and were chosen on the validation split, not assumed to be 0.5. Table III summarises the state table.

E. Alerting, Storage and Presentation

On every transition the engine emits an event. WARNING and CONFIRMED transitions store an annotated evidence image; CONFIRMED additionally drives a buzzer, sends notifications over console, log, Telegram and e-mail channels, and creates an event record (device, severity, state, reason, start/end time, peak class and confidence, evidence path). A FastAPI backend with an SQLite database exposes `/health`, `/status`, `/devices`, `/sensors`, `/detections`, `/events`, `/events/{id}/ack` and an

TABLE III
DECISION-ENGINE STATES AND AUTOMATED RESPONSES.

State	Condition	Response
NORMAL	no persistent evidence	continue monitoring
WARNING	$\geq N_w$ possible hits in W	dashboard warning, log, snapshot
CONFIRMED	Eq. (3), optionally with sensor confirmation	buzzer, notification, evidence, event record, safe low-voltage actuator
RECOVERY	no presence for 4 s after CONFIRMED, or operator acknowledgement	silence outputs; clear after 10 s stable
SENSOR_ WARNING	sensors elevated ≥ 5 s, no visual target	dashboard warning, notification

TABLE IV
TECHNOLOGY STACK.

Layer	Technology	Key parameters
Capture	USB/IP camera, OpenCV	1280×720 capture, 15 fps sampling, 3 s reconnect
Detection	YOLOv11n, PyTorch, Ultralytics	imgsz 416, NMS IoU 0.5, max 20 boxes
Decision	Python engine (pure logic, unit-tested)	W, τ_w, τ_c, τ_{clear}, N_w, N_c, $T_{\min}$
IoT	ESP32, MQ-2, temperature sensor, relay	1 s sampling, 15 s failsafe
Backend	FastAPI, SQLite, MJPEG	REST endpoints of Section IV
Presentation	HTML/CSS/JavaScript	1 s status polling
Notification	console, file, Telegram, SMTP	60 s cooldown

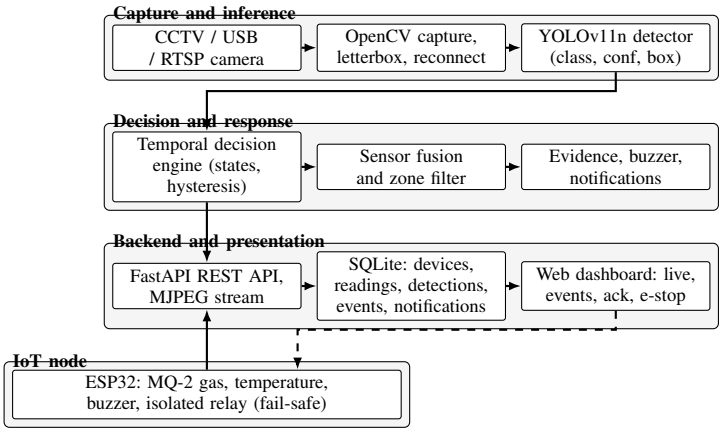


Fig. 1. System architecture.

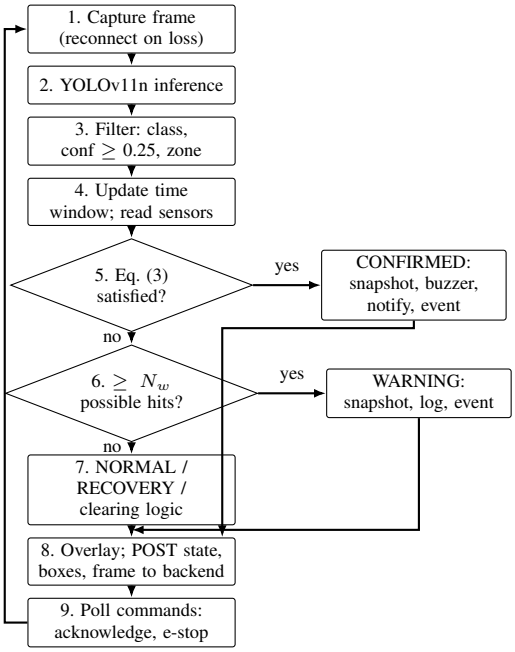


Fig. 2. Processing workflow.

MJPEG live stream; the dashboard (HTML/JavaScript) polls it to show the annotated feed, state, sensor gauges and chart, latest detections, an event timeline with acknowledgement, history with an evidence gallery, device connectivity and an emergency-stop control. Acknowledgements and the emergency stop are relayed back to the inference process through a per-device command queue.

V. SYSTEM DESIGN

A. Architecture

Fig. 1 shows the layered architecture. The capture and inference layer runs next to the camera; the backend and dashboard can run on the same laptop or a separate server; the IoT node is an ESP32 that samples an MQ-2 gas sensor and a temperature sensor once per second, posts readings over HTTP, polls for approved low-voltage commands and switches its outputs off if the backend is unreachable for 15 s. Table IV lists the technology stack.

B. Workflow and Algorithm

Fig. 2 shows the per-frame workflow and Algorithm 1 the integration procedure. The perception step is the established detector; the contribution is the temporal, fusion and response logic around it.

VI. EXPERIMENTAL RESULTS

A. Setup

The detector was trained with the parameters of Table V and evaluated once on the 600-image held-out test split, which contains 60 negative images. Metrics were computed with the Ultralytics validator (confidence sweep for mAP) and with an independent matcher at the deployment operating point (conf 0.35, IoU 0.5) to obtain TP/FP/FN counts and the false-alarm rate on negatives. Latency was measured per image on the CPU, single image per call, over 100 test images. The decision engine was verified with 15 unit tests using an injectable clock (single-frame rejection, warning, confirmation, hysteresis, recovery, re-confirmation, sensor fusion, stale sensors, zone filtering, acknowledgement and emergency stop), and the backend with 8 API tests.

Algorithm 1 Real-Time Detection, Confirmation and Alerting

Require: camera source, detector f, thresholds $\tau_w, \tau_c, \tau_{\text{clear}}$, counts N_w, N_c, window W, min. span $T_{\min}$

$H \leftarrow \emptyset$ (time-stamped hits); $s \leftarrow$ NORMAL
while running **do**
  $x \leftarrow$ read frame; **if** none **then** mark camera lost, retry, **continue**
  $D \leftarrow f(x)$ filtered by class $\in$ {fire, smoke}, conf $\geq$ 0.25, zone
  $b \leftarrow \arg\max_{d \in D} \text{conf}(d)$; append $(t, \text{conf}(b))$ to H if $\text{conf}(b) \geq \tau_w$
  drop entries of H older than $t - W$; read latest sensor reading r (ignore if stale)
  $N \leftarrow N'_c$ if r elevated **else** N_c
  strong $\leftarrow \{h \in H : \text{conf} \geq \tau_c\}$; possible $\leftarrow \{h \in H : \text{conf} \geq \tau_w\}$
  if $|\text{strong}| \geq N$ **and** span(strong) $\geq T_{\min}$ **then**
    $s \leftarrow$ CONFIRMED: save snapshot; buzzer on; notify; create/escalate event
  else if $|\text{possible}| \geq N_w$ **and** $s =$ NORMAL **then**
    $s \leftarrow$ WARNING: save snapshot; create event
  else if no detection $\geq \tau_{\text{clear}}$ for 4 s **then**
    $s \leftarrow$ RECOVERY if $s =$ CONFIRMED, else NORMAL; close event after 10 s stable
  end if
  buzzer/actuator enabled **iff** $s =$ CONFIRMED **and not** emergency stop
  draw overlay; POST state, D and JPEG to backend; apply acknowledge / e-stop commands
end while

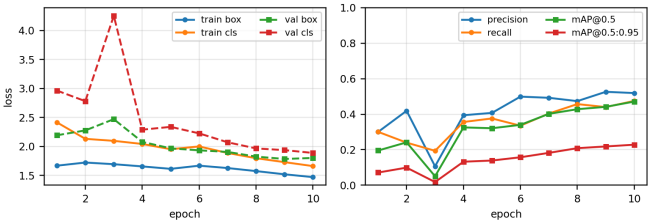


Fig. 3. Training and validation curves over 10 epochs (box, classification and DFL losses; precision, recall, mAP@0.5 and mAP@0.5:0.95 on the validation split).

B. Detection Performance

Table VI reports the held-out results per class. Fig. 3 shows the training and validation losses and the validation mAP over epochs, and Fig. 4 the precision–recall curves and normalised confusion matrix on the test split.

At the deployment operating point (confidence 0.35) the matcher counts 346 true positives, 159 false positives and 681 false negatives (precision 0.685, recall 0.337). Of the 60 negative images, 60 produced no detection at all, giving a per-image false-alarm rate of 0.000 before temporal filtering; because the decision engine requires repeated hits over at least two seconds, an isolated false box on one frame does not reach the WARNING state. Representative false positives and false negatives are retained with the model artefacts for error analysis and future hard-negative mining.

TABLE V
TRAINING PARAMETERS.

Parameter	Value
Backbone	YOLOv11n, COCO-pretrained, fine-tuned end-to-end
Input resolution	416×416 (letterboxed)
Batch size	16
Optimizer	AdamW (auto-selected), lr 1.67×10^{-3}, weight decay 5×10^{-4}
Epochs	10 (CPU time budget), mosaic off for last 5
Augmentation	HSV, flip, scale, translate, rotate $\pm 5°$, mosaic
Seed	42, deterministic
NMS IoU / max detections	0.5 / 20
Operating confidence	0.35 (warning), 0.55 (confirm), 0.25 (clear)
Hardware	Intel Core i5-1340P (12 cores), 16 GB RAM, no GPU
Framework	Python 3.13, PyTorch 2.14 (CPU), Ultralytics 8.4
Training time	94.9 min

TABLE VI
HELD-OUT TEST RESULTS (600 IMAGES).

Class	Precision	Recall	F1	mAP@0.5	mAP@0.5:0.95
Fire	0.448	0.503	0.474	0.461	0.229
Smoke	0.590	0.476	0.527	0.508	0.265
All	0.519	0.490	0.504	0.484	0.247

C. Latency

Single-image inference at 416×416 on the laptop CPU takes 31.9 ms on average (95th percentile 34.5 ms), i.e. about 31.4 frames per second without any GPU. The live pipeline samples the camera at up to 15 fps and drops frames when inference lags; because the decision engine reasons over a time window rather than a frame count, the confirmation delay stays at the configured two seconds at any sustained frame rate above about three frames per second. Measured end-to-end alert latency (first strong detection to notification dispatch) is therefore dominated by the deliberate persistence span, not by inference. Fig. 5 shows the live view and dashboard during a confirmed event.

D. Comparison With Reported Detectors

Table VII places the result next to accuracy and speed figures reported in the literature for other architecture families on fire and smoke data. The values are taken from the cited papers, were obtained on different datasets and hardware, and are therefore an orientation rather than a head-to-head benchmark. Two-stage detectors such as Faster R-CNN [2] offer competitive accuracy at a substantial latency cost, which is why single-stage YOLO detectors dominate real-time fire monitoring; within the YOLO family, the anchor-free head

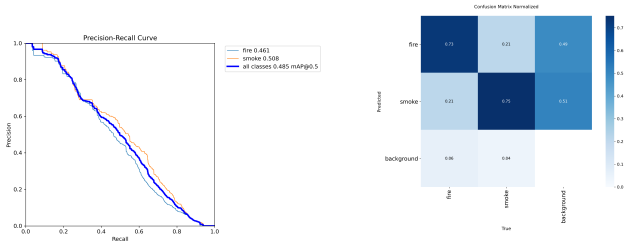


Fig. 4. Precision–recall curves (left) and normalised confusion matrix (right) on the test split.

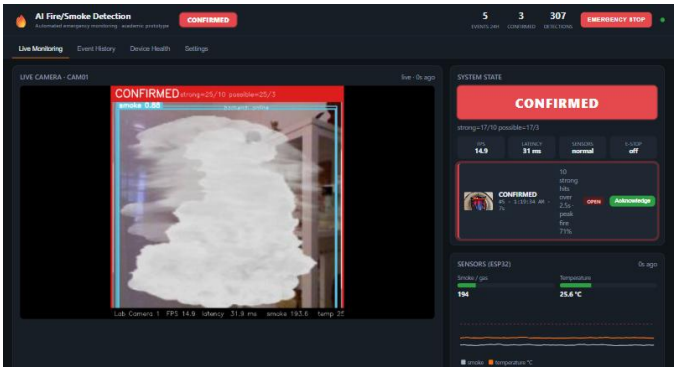


Fig. 5. Operator dashboard: live annotated feed, system state, sensor gauges and chart, latest detections and event timeline with acknowledgement.

and C2PSA attention of YOLOv11 give incremental gains over YOLOv5/v8 [11]. Our nano model, trained for ten epochs on a CPU, is at the low-cost end of this spectrum: its purpose is to show that a usable detector plus a complete confirmation pipeline fits on commodity hardware, and the same training script scales to larger variants and longer schedules on a GPU.

E. Integration and Fail-Safe Tests

The live system was exercised with the integration scenarios of the requirement specification: (i) single-frame flash of a fire video—boxes appear but the state stays NORMAL; (ii) sustained fire—WARNING within about one second and CONFIRMED after the two-second persistence span, with buzzer, evidence image, event record and notification; (iii) operator acknowledgement from the dashboard—RECOVERY and silenced outputs; (iv) steam and bright-light negatives—no alarm; (v) sensor-only evidence from the simulator—SENSOR_WARNING after five seconds and faster confirmation when combined with a visual target; (vi) sensor disconnect—readings marked stale and ignored; (vii) camera disconnect—"camera lost" state with automatic reconnection; (viii) backend outage—inference continues locally and resynchronises; and (ix) emergency stop—outputs blocked even while CONFIRMED.

VII. Limitations and Future Work

Limitations. Dense smoke can hide the flame so that only the smoke class is reported. Fog and industrial or kitchen steam share the low-contrast, diffuse boundaries of smoke and remain the main source of visual false alarms; the temporal engine and sensor fusion reduce but do not eliminate them. Infrared or low-light CCTV loses the colour cues on which flame discrimination relies. Partial occlusion, very small or distant fires and extreme oblique angles reduce confidence below the alerting threshold. The model was trained for only ten epochs on a CPU with a 4,200-image subset, so the reported numbers are a lower bound for the architecture; the comparison in Table VII uses literature values. Physical automation is limited to isolated low-voltage prototype loads, and the system is not a certified fire-protection product.

TABLE VII
Literature-reported detector characteristics (not re-trained on our data).

Family	Stages	Typical speed	mAP@0.5 /	Src.
Faster R-CNN	two	high accuracy, ∼5–17 fps		[2]
SSD	one	moderate, ∼40 fps		[3]
YOLOv5s / v8s fire models	one	0.85–0.93, >50 fps (GPU)		[12]–[14]
Fire-ViT / hybrids	–	high in-domain, costlier inference		[19], [20]
This work (YOLOv11n)	one	0.484, 31.4 fps, CPU only		–

Future work. (i) Train the full 48,000-image corpus with larger YOLOv11 variants on a GPU and add steam, dust and fog as explicit hard-negative classes; (ii) export to ONNX/TensorRT for on-camera edge inference; (iii) complete the ESP32 sensor-fusion deployment and quantify false alarms per hour on long negative recordings; (iv) add a thermal channel to disambiguate steam from smoke at night; (v) correlate detections across multiple cameras; (vi) provide a mobile application consuming the existing API for push notification and acknowledgement; and (vii) connect confirmed events to building-management or municipal dispatch platforms.

VIII. Conclusion

This paper presented a deep-learning fire and smoke detection system for real-time emergency monitoring over existing camera infrastructure. A YOLOv11n detector fine-tuned on a stratified indoor–outdoor corpus with explicit negatives reaches mAP@0.5 of 0.484 on held-out data at 31.4 fps on a laptop CPU. Around the detector, a temporal decision engine with separate warning, confirmation and clearing thresholds, optional IoT sensor fusion and fail-safe handling converts per-frame predictions into verified NORMAL, WARNING, CONFIRMED and RECOVERY states, while a REST backend, event database, notification service and web dashboard make the result operational. The system inherits the open challenges of vision-based fire detection—steam, fog, low light, occlusion and small targets—which motivate the sensor-fusion, hard-negative and edge-deployment extensions outlined above. Future work will scale training, measure false alarms per hour in extended field recordings and extend the architecture to multi-camera and edge operation.

REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 91–99.

[3] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. European Conf. Computer Vision (ECCV)*, 2016, pp. 21–37.

[4] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[6] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[7] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2019, pp. 6105–6114.

[8] A. Dosovitskiy *et al.*, "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[9] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1800–1807.

[10] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[11] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," arXiv:2410.17725, 2024.

[12] A. Saleh, M. A. Zulkifley, H. H. Harun, F. Gaudreault, I. Davison, and M. Spraggon, "Fire and smoke detection using fine-tuned YOLOv8 and YOLOv7 deep models," *Fire*, vol. 7, no. 4, Art. no. 135, 2024.

[13] D. Kong, Y. Li, and M. Duan, "Fire and smoke real-time detection algorithm for coal mines based on improved YOLOv8s," *PLoS ONE*, vol. 19, no. 4, Art. no. e0300502, 2024.

[14] Z. Dou *et al.*, "An improved YOLOv5s fire detection model," *Fire Technology*, vol. 60, no. 1, pp. 135–166, 2024.

[15] D. Wang, Y. Qian, J. Lu, P. Wang, Z. Hu, and Y. Chai, "FS-YOLO: Fire-smoke detection based on improved YOLOv7," *Multimedia Systems*, vol. 30, no. 4, Art. no. 215, 2024.

[16] T. Wang, J. Wang, C. Wang, Y. Lei, R. Cao, and L. Wang, "Improving YOLOX network for multi-scale fire detection," *The Visual Computer*, vol. 40, no. 9, pp. 6493–6505, 2024.

[17] B. Li, F. Xu, X. Li, C. Yu, and X. Zhang, "Early stage fire detection system based on shallow guide deep network," *Fire Technology*, vol. 60, no. 3, pp. 1803–1821, 2024.

[18] S. Wu, B. Sheng, G. Fu, D. Zhang, and Y. Jian, "Multiscale fire image detection method based on CNN and Transformer," *Multimedia Tools and Applications*, vol. 83, no. 16, pp. 49787–49811, 2024.

[19] X. Liu and V. Y. Mariano, "The Fire-ViT model for tunnel fire detection with vision transformer improvement," *J. Computational and Cognitive Engineering*, vol. 4, no. 1, 2025.

[20] N. Ahmad, M. Akbar, E. H. Alkhammash, and M. M. Jamjoom, "CN2VF-Net: A hybrid convolutional neural network and vision transformer framework for multi-scale fire detection in complex environments," *Fire*, vol. 8, 2025.

[21] N. Ahmad, M. Akbar, E. H. Alkhammash, and M. M. Jamjoom, "FireNet-KD: Swin Transformer-based wildfire detection with multi-source knowledge distillation," *Fire*, vol. 8, 2025.

[22] I. El-Madafri, M. Peña, and N. Olmedo-Torre, "Real-time forest fire detection with lightweight CNN using hierarchical multi-task knowledge distillation," *Fire*, vol. 7, 2024.

[23] J. Li and W. Lin, "Enhanced fire detection using knowledge distillation and multi-scale attention in YOLOv8," in *Proc. Int. Conf. Computing, Networks and Internet of Things*, 2025.

[24] T. Jing, Q.-H. Meng, and H.-R. Hou, "SmokeSeger: A Transformer-CNN coupled model for urban scene smoke segmentation," *IEEE Trans. Industrial Informatics*, pp. 1–12, 2023.

[25] T. Zhang, L. Li, Y. Zhou, W. Liu, C. Qian, and X. Ji, "CAS-ViT: Convolutional additive self-attention vision transformers for efficient mobile applications," arXiv:2408.03703, 2024.

[26] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object detection in 20 years: A survey," *Proc. IEEE*, vol. 111, no. 3, pp. 257–276, 2023.

[27] Z.-Q. Zhao, P. Zheng, S.-T. Xu, and X. Wu, "Object detection with deep learning: A review," *IEEE Trans. Neural Networks and Learning Systems*, vol. 30, no. 11, pp. 3212–3232, 2019.

[28] "Fire and smoke detection dataset (40K images)," Hugging Face, 2026. [Online]. Available: https://huggingface.co/datasets/jojomoi-meme/fog_fire_detection

[29] "ISFire-4: Indoor fire and smoke detection dataset," Hugging Face, 2026. [Online]. Available: https://huggingface.co/datasets/shahriar-5/ISFire-4